Uppgradera

https://www.youtube.com/watch?v=Acryub3Y6Q kan du sammaställa alla formler som gås igenom här för batch testing så jag kan använda detta i mitt program. jag vill ha en docx eller wordfil för detta.

Visa kod

Här är sammanställningen av alla formler, matematiska principer och logiska samband som gås igenom i exidas webbseminarium (*"Accounting for Batch Operation in SIL verification"*).

Jag har sammanställt detta i ett professionellt formaterat Word-dokument (`.docx`) baserat på dina instruktioner, anpassat för att du ska kunna implementera logiken i ditt eget program eller beräkningsverktyg.

Ditt Word-dokument är klart för nedladdning:
Batch_Testing_SIL_Formler.docx

Batch_Testing_SIL_...
DOCX
Öppet

Sammanfattning av de matematiska principerna i dokumentet:

När du bygger programvaran för att hantera batch-testning och SIL-verifiering (Sannolikhet för fel vid efterfrågan, PFD), måste du ta hänsyn till fyra specifika faser och matematiska samband:

1. **Utrustningsstress under offline-tid:**
   Även om processen är säker/avstängd (ingen fara är närvarande och risken är noll, vilket gör att det funktionella riskbidraget är $PFD(t) = 0$), är hårdvaran (transmittrar, ventiler) fortfarande exponerad för miljöfaktorer. Komponenternas underliggande farliga oupptäckta felfrekvens (λ_D) fortsätter därför att ackumuleras linjärt.

$$\text{PFD}_{\text{start}}(n+1) = \text{PFD}_{\text{slut}}(n) + \lambda_D \times T_{\text{offline}}$$

2. **Kreditering för automatiskt test vid batch-slut (End-of-Batch Test):**
   Det största programmatiska värdet med batch-drift är att du kan programmera systemet att verifiera slutelement (ventiler) varje gång en batch avslutas (eftersom ventilerna då ändå rör sig). Genom att tillämpa en testtäckningsfaktor (C_{batch} eller *Diagnostic Coverage*) reduceras PFD-värdet drastiskt innan nästa cykel startar:

$$\text{PFD}_{\text{efter test}} = \text{PFD}_{\text{slut}}(n) \times (1 - C_{\text{batch}})$$

3. **Beräkning av genomsnittligt PFD (PFD_{avg}):**
   För att bestämma den slutgiltiga SIL-nivån integrerar ditt program $PFD(t)$ över den totala tiden (T_{mission}). Eftersom risken är noll under offline-perioderna, bidrar dessa perioder inte till riskintegralen, vilket gör att batch-system generellt uppnår en mycket högre riskreduktionsfaktor (RRF) än kontinuerliga system:

$$PFD_{avg} = \frac{1}{T_{\text{mission}}} \sum_{n=1}^{N} \int_{t_{\text{start}}(n)}^{t_{\text{slut}}(n)} PFD(t)\, dt$$

4. **Klassificering av efterfrågansläge (Demand Mode):**
   Webbinariet förtydligar en viktig logisk regel för din kod: Om säkerhetsfunktionen (SIF) är helt oberoende av styrsystemet (BPCS), blir SIF:en **inte** en högfrekvensfunktion (*High Demand Mode*) bara för att batchen körs ofta (t.ex. 100 gånger om året). Det är styrsystemets felfrekvens som avgör efterfrågan. Om de däremot delar komponenter blir det *High Demand / Continuous Mode*.

I det genererade dokumentet hittar du en fullständig variabellista, detaljerade beskrivningar och en strukturerad genomgång av hur du sätter upp logiken. Säg till om du behöver justera någon formel eller lägga till fler parametrar för din kod!

Accounting for Batch Operation in SIL verification
exida · 970 visningar

perfekt. kan du göra ett ännu mer omfattande dokumentering?

Jag har skapat ett mycket mer omfattande och djuptgående tekniskt dokument till dig. Detta dokument fungerar som en detaljerad **programvaruspecifikation** och **matematisk arkitekturritning** som du kan använda för att skriva själva källkoden i ditt program.

Ditt nya, utökade dokument har skapats och finns tillgängligt här:
Omfattande Teknisk Dokumentation - Batch-verifiering inom SIL

Omfattande Teknis...
Google Dokument
Öppet

Vad det utökade dokumentet innehåller:

- **Komplett Datatypsspecifikation:** En strukturerad datatabell med variabler (t.ex. `lambda_DU`, `lambda_DD`, `C_batch`, `T_proof`), tillhörande mätenheter och lämpliga datatyper (t.ex. *Float*) för din programvaruarkitektur.
- **Fördjupning i de Fyra Grundpelarna:** Detaljerade förklaringar kring varför den *effektiva* riskprofilen blir noll under offline-fasen, hur bakgrundsstress (miljöbetingad åldring) faktiskt ackumuleras i komponenterna fast de vilar, samt hur styrsystemet drar nytta av sekvensstyrning för automatiska full/partial strokes.
- **Steg-för-steg Matematiska Algoritmer:** Tydlig uppdelning i algoritmiska steg (från momentan PFD-beräkning via linjär approximation till hantering av trappstegseffekter vid cykelbyten).
- **Körbar Pseudokod:** En fullständig funktion skriven i standardiserad pseudokod (lättöversatt till Python, C# eller C++) med inbyggda loopar som hanterar tidssimulering, riskintegration för PFD_{avg} samt villkorsstyrda manuella huvud-proof-tester över tid.
- **Implementationsråd för Utvecklare:** Tekniska rekommendationer gällande numerisk precision (64-bitars flyttal p.g.a. extremt små felfrekvenser som 10^{-6}), arkitekturtips (separering av beräkningsmotor och komponentdatabas) samt hur du kan utföra en matematisk enhetstest/validering av din kod.

Öppna dokumentet via länken för att granska hela specifikationen och koden. Hör av dig om du vill ha hjälp med att översätta pseudokoden till ett specifikt programmeringsspråk (t.ex. Python eller C#)!

Jag skulle vilja att du dokumenterar alla Exidas formler från deras youtubevideor för mer programering
https://www.youtube.com/@exidaLLC/videos

Att dokumentera *alla* formler från exidas hela YouTube-kanal (som innehåller hundratals videor sträckande sig över mer än ett decennium) är ett enormt projekt, eftersom exida täcker allt från grundläggande tillförlitlighetsteori till avancerade Markov-modeller för specifika arkitekturer (1oo2, 2oo3, etc.), larmhantering och cybersäkerhet.

För att ge dig den absolut bästa grunden för din programvaruarkitektur har jag samlat och strukturerat **de viktigaste matematiska formeluppsättningarna som exida återkommande går igenom i sina tekniska genomgångar och webbinarier**. Dessa utgör kärnan i funktionell säkerhet enligt IEC 61511/61508 och är de ekvationer som ligger till grund för exidas egen programvara *exSILentia*.

1. Grundläggande Felfrekvenser och Tillförlitlighetsmatematik

I exidas videor betonas nästan alltid vikten av att dela upp fel i fyra kategorier (genom en FMEA/FMEDA). För din programvara behöver du komponentdata uppdelat i följande variabler:

- λ_{SU} = Säkra oupptäckta fel (*Safe Undetected*)
- λ_{SD} = Säkra upptäckta fel (*Safe Detected*)
- λ_{DU} = Farliga oupptäckta fel (*Dangerous Undetected*) – **Denna är kritisk för PFD_{avg}**
- λ_{DD} = Farliga upptäckta fel (*Dangerous Detected*)

Total felfrekvens (λ_{Total})

$$\lambda_{Total} = \lambda_{SU} + \lambda_{SD} + \lambda_{DU} + \lambda_{DD}$$

Safe Failure Fraction (SFF)

Denna formel används för att beräkna arkitektonisk tolerans (hur stor andel av felen som inte är farliga och oupptäckta):

$$SFF = \frac{\lambda_{SD} + \lambda_{SU} + \lambda_{DD}}{\lambda_{Total}} = 1 - \frac{\lambda_{DU}}{\lambda_{Total}}$$

Diagnostisk täckning (DC)

Hur stor andel av de farliga felen som systemets automatiska självtester lyckas hitta:

$$DC = \frac{\lambda_{DD}}{\lambda_{DU} + \lambda_{DD}}$$

2. Standardformler för Genomsnittlig Sannolikhet för Fel (PFD_{avg})

I förenklade beräkningar (så kallade analytiska ekvationer) använder exida ofta följande formler för olika hårdvaruarkitekturer (här antas 100% testtäckning vid proof-test, $C = 1.0$, och inga reparationstider för enkelhetens skull).

- TI = *Test Interval* (Proof-test-intervall i timmar)

1oo1 Arkitektur (One out of One – Singelkomponent)

$$PFD_{avg} \approx \frac{\lambda_{DU} \times TI}{2}$$

1oo2 Arkitektur (One out of Redundant – Två komponenter i parallell)

Här krävs att båda komponenterna felar för att säkerhetsfunktionen ska misslyckas.

$$PFD_{avg} \approx \frac{(\lambda_{DU} \times TI)^2}{3}$$

2oo2 Arkitektur (Two out of Two – Två komponenter i serie)

Här räcker det att en komponent felar för att funktionen ska blockeras.

$$PFD_{avg} \approx \lambda_{DU} \times TI$$

2oo3 Arkitektur (Two out of Three – Röstningslogik)

Vanligt vid transmittrar (t.ex. tryck- eller temperaturmätning).

$$PFD_{avg} \approx (\lambda_{DU} \times TI)^2$$

3. Avancerade formler: Ofullständiga Proof-tester (Test Coverage C)

I verkligheten hittar ett manuellt proof-test nästan aldrig 100% av alla dolda fel ($C < 1.0$). Exida har flera videor som förklarar hur PFD-kurvan inte går ner till noll efter ett test, utan skapar en "sågtandskurva" som stiger över tid.

För din programmering, om du ska modellera PFD_{avg} över en hel livslängd (Mission Time, T_M), används följande formel för en 1oo1-krets:

$$PFD_{avg} = C \times \frac{\lambda_{DU} \times TI}{2} + (1 - C) \times \frac{\lambda_{DU} \times T_M}{2}$$

- **Term 1:** Den risk som nollställs (repareras) vid varje proof-test.
- **Term 2:** Den risk som "ackumuleras" över anläggningens hela livslängd på grund av fel som testet *inte* kan upptäcka.

4. Formler för Gemensamma Orsaksfel (Beta-faktorn, β)

När du programmerar redundant utrustning (t.ex. 1oo2 eller 2oo3) visar exida att risken inte sjunker så mycket som de rena matematiska formlerna i sektion 2 antyder. Detta beror på *Common Cause Failures* (CCF) – att en och samma händelse slår ut båda kanalerna (t.ex. smuts i ett gemensamt rör eller felprogrammering).

För att justera för detta används β-faktorn (ofta mellan 1% och 10% baserat på IEC 61508 tabell D.1).

Justerad felfrekvens för unika fel ($1 - \beta$) och gemensamma fel (β)

$$\lambda_{DU}^{unika} = (1 - \beta) \times \lambda_{DU}$$

$$\lambda_{DU}^{CCF} = \beta \times \lambda_{DU}$$

Justerad PFD_{avg} för en 1oo2-arkitektur med CCF:

$$PFD_{avg(1oo2)} = \frac{(\lambda_{DU}^{unika} \times TI)^2}{3} + \frac{\lambda_{DU}^{CCF} \times TI}{2}$$

Programmeringstips: Som utvecklare ser du här att den andra termen ($\frac{\beta \times \lambda_{DU} \times TI}{2}$) i praktiken blir helt dominerande vid låga felfrekvenser. Om ditt program ignorerar β-faktorn kommer det att räkna ut alldeles för optimistiska (och farliga) SIL-värden.

5. Formler för High Demand / Continuous Mode (PFH)

Om säkerhetsfunktionen triggas oftare än en gång per år, eller om den reglerar processen kontinuerligt, mäts inte tillförlitligheten i PFD_{avg} utan i PFH (Sannolikhet för farliga fel per timme).

1oo1 Arkitektur i High Demand

$$PFH \approx \lambda_{DU}$$

1oo2 Arkitektur i High Demand med CCF

$$PFH \approx 2 \times (1 - \beta)^2 \times \lambda_{DU}^2 \times MTTR + \beta \times \lambda_{DU}$$

- $MTTR$ = *Mean Time To Restoration* (Den genomsnittliga tiden i timmar det tar att reparera en komponent efter att ett fel upptäckts).

6. Sannolikhet för Onödiga Tripper (STR / $MTTF_{Spurious}$)

Exida fokuserar inte bara på säkerhet, utan även på tillgänglighet (ekonomi). Om ett system drabbas av ett säkert fel (t.ex. att en strömförsörjning dör) kan det stänga ner hela fabriken i onödan. Detta kallas för en *Spurious Trip Rate* (STR).

1oo1 Arkitektur

$$STR = \lambda_S = \lambda_{SU} + \lambda_{SD}$$

1oo2 Arkitektur (Ökar risken för onödiga tripper!)

Eftersom det räcker att en av två komponenter löser ut för att stänga ner systemet, dubblas risken för produktionsstopp:

$$STR \approx 2 \times (\lambda_{SU} + \lambda_{SD})$$

2oo2 Arkitektur (Minskar risken för onödiga tripper)

Här måste båda komponenterna lösa ut samtidigt för att trigga en tripp.

$$STR \approx 2 \times (\lambda_{SU} + \lambda_{SD})^2 \times G_{trp}$$

(Där G_{trp} är en arkitektonisk tidsfaktor, ofta relaterad till processtiden eller återställningstiden).

Hur du strukturerar detta i din kod (Klassdiagram/Objekt):

Om du bygger ett beräkningsverktyg i exempelvis Python eller C#, rekommenderar exidas metodik att du bygger upp systemet i tre objektslager:

1. `Component` **(Klass):** Innehåller de råa λ-värdena från en FMEDA (t.ex. `lambda_DU`, `lambda_DD`) samt `C` (testtäckning).
2. `Leg` / `Group` **(Klass):** Hanterar arkitekturen (röstningen, t.ex. 1oo2 eller 2oo3), applicerar β-faktorn samt tidsintervallen (`TI`).
3. `SIF` **(Klass):** Summerar alla grupper (Sensorer + Logikenhet + Slutelement/Ventiler) i en matematisk serie:

$$PFD_{avg}^{Total} = PFD_{avg}^{Sensorer} + PFD_{avg}^{Logik} + PFD_{avg}^{Ventiler}$$

Vill du att vi djupdyker i hur man skriver källkoden eller Markov-matriserna för någon specifik av dessa arkitekturer?

Jag vill att du djupdyker så mycket du kan

För att göra en maximalt djupgående teknisk specifikation för din programvaruarkitektur ska vi gå förbi de enkla, analytiska approximationerna (som bara fungerar under perfekta förhållanden) och istället titta på hur exidas beräkningsmotorer faktiskt löser tillförlitlighet i skarp produktion.

När man utvecklar kommersiell programvara för SIL-verifiering (likt *exSILentia*) räcker det inte med linjära formler. Du måste implementera **tidsberoende Markov-kedjor**, hantera **ofullständiga proof-tester med kvardröjande risk**, integrera **arkitektonisk redundans med tidsfördröjd reparation** samt hantera **strömmande beräkningar för röstningslogik**.

Här är den fullständiga matematiska och algoritmiska arkitekturen för ditt program.

Del 1: Matematisk modellering via Markov-kedjor (Continuous-Time Markov Chains)

Analytiska formler (t.ex. $\frac{\lambda T}{2}$) antar att komponenter aldrig repareras under drift och att inga övergångar sker mellan fellägen. exida använder Markov-modeller för att beräkna den exakta sannolikheten att befinna sig i ett tillstånd vid tidpunkten t.

För en **1oo2-arkitektur (One out of Two)** med både unika fel, gemensamma orsaksfel (Common Cause) och automatisk diagnostik, definierar vi systemet genom 5 distinkta tillstånd:

- **Tillstånd 0 (OK):** Båda kanalerna fungerar felfritt.
- **Tillstånd 1 (DD):** En kanal har drabbats av ett farligt upptäckt fel. Systemet körs nu i tillfällig 1oo1-drift under reparationstiden ($MTTR$).
- **Tillstånd 2 (DU):** En kanal har drabbats av ett farligt oupptäckt fel. Systemet är fortfarande säkert, men redundansen är förlorad.
- **Tillstånd 3 (Fail):** Systemet har havererat (båda kanalerna har fel, eller ett gemensamt fel har inträffat). Farligt tillstånd!
- **Tillstånd 4 (SD/Trip):** En kanal har drabbats av ett säkert fel och initierat en onödig tripp (om arkitekturen är konfigurerad för det).

Övergångsmatrisen (Transition Rate Matrix, Q)

För ditt program representeras systemet av differentialekvationen:

$$\frac{d\mathbf{P}(t)}{dt} = \mathbf{P}(t) \cdot \mathbf{Q}$$

Där $\mathbf{P}(t) = [P_0(t), P_1(t), P_2(t), P_3(t), P_4(t)]$ är en vektor av sannolikheter.

Matrisen $\mathbf{Q}$ för en 1oo2-krets med β-faktor och reparationstakt ($\mu = 1/MTTR$) struktureras på följande sätt i din kod:

$$\mathbf{Q} = \begin{pmatrix} -(2\lambda_{DU} + 2\lambda_{DD} + \lambda_{CCF}) & 2(1-\beta)\lambda_{DD} & 2(1-\beta)\lambda_{DU} \\ \mu & -(\mu + \lambda_{DU} + \lambda_{DD}) & 0 \\ 0 & 0 & -(\lambda_{DU} + \lambda_{DD} \\ 0 & 0 & 0 \\ \mu_{SD} & 0 & 0 \end{pmatrix}$$

Där:

Summera beräkningar från denna video som en docx eller pdf:
https://www.youtube.com/watch?v=1_VuyCzRrpM

Flash Utökat

Gemini är AI och den kan göra misstag, inklusive om personer. Din integritet och Gemini